

PROJECT REPORT

COURSE CODE: 24ADI204

COURSE TITLE: DATA SCIENCE AND VISUALIZATION

Weather-Driven Renewable Energy Analysis

SUBMITTED BY

K S. NAGUL NISOK (25BAD063)

V. NIDHISH (25BAD067)

B. NIRMAL (25BAD068)

S. NIVESH (25BAD073)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

KUMARAGURU COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE - 641 049

INTRODUCTION:

The project "**Weather-Driven Renewable Energy Analysis**" focuses on studying how daily weather conditions affect solar and wind power generation across five locations in Tamil Nadu. Renewable energy output changes constantly with temperature, cloud cover, humidity, rainfall and wind speed, and understanding these links is essential for planning reliable power supply.

The project uses a daily weather and energy log that records temperature, humidity, wind speed, solar irradiance, cloud cover, rainfall and air pressure alongside solar energy, wind energy, total renewable energy and energy demand. Exploratory Data Analysis (EDA) is performed to understand how these variables behave and relate to one another, after first cleaning a hidden data-quality problem in the file.

The analysis aims to identify the weather factors most strongly associated with renewable power output, and to hand over a clean, well-understood dataset that is ready for feature engineering and modelling in the following weeks.

Domain:**Renewable Energy / Environmental Data Analytics**

The renewable energy sector generates power from natural sources such as sunlight and wind, both of which vary constantly with local weather. Unlike conventional power plants, solar and wind output cannot be switched on at will - it depends directly on the conditions on a given day.

Accurately understanding this weather-to-output relationship is important because it helps grid operators plan for shortfalls, helps planners decide where new solar or wind capacity will perform best, and helps identify which weather variables are the strongest and most reliable predictors of power generation.

In this project, a **daily renewable energy and weather dataset** covering five Tamil Nadu locations is used to study temperature, humidity, wind speed, solar irradiance, cloud cover, rainfall, air pressure, weather condition, and the resulting solar, wind, total renewable and demand figures.

Problem Statement

Renewable energy planners need to know how much solar and wind power to expect from the weather, but the raw data log used for this analysis is not directly trustworthy. Failed sensor readings in the log are stored using the code -999 instead of being left blank, and pandas reads this code as a normal number.

As a result, any summary statistic computed on the raw file is wrong: for example, the mean wind speed comes out to -2.71 km/h and the minimum recorded temperature comes out to -999 degrees Celsius, both of which are physically impossible.

The problem is therefore twofold: first, to detect and correctly handle this hidden missing-value code (along with eight duplicate rows and a number of very high values), and second, to use Exploratory Data Analysis to discover which weather variables are most strongly linked to solar power, wind power and total renewable output.

The project aims to use systematic data cleaning followed by EDA to uncover reliable, meaningful patterns that can guide renewable-energy planning and support later feature engineering and modelling.

Problem Abstract

Reliable renewable-energy planning depends on understanding exactly how weather drives solar and wind power output. This project, titled "Weather-Driven Renewable Energy Analysis," works with 510 days of weather and energy readings from five locations in Tamil Nadu to uncover these patterns.

The project begins with data preprocessing: the -999 sentinel code used for failed sensor readings is converted to a proper missing value, the resulting gaps are filled using the column median, and duplicate rows and unusually high or low values are identified and reviewed. Exploratory Data Analysis is then carried out to examine the distribution of each weather and energy variable, and to compare pairs of variables for meaningful relationships.

Correlation analysis is used to measure the strength of the links between weather conditions and power output, most notably between cloud cover and solar irradiance, and between wind speed and total renewable energy. The resulting insights identify the weather variables most worth carrying forward into feature engineering and predictive modelling in the coming weeks.

OBJECTIVES:

1. To understand how weather conditions influence solar and wind renewable energy output.
2. To load and manipulate the renewable energy weather dataset using Pandas and NumPy.
3. To profile the dataset - its shape, column types, and summary statistics - and produce a First Impression Report.
4. To detect the -999 sentinel code used for failed sensor readings and convert it into proper missing values.
5. To measure how much data is missing in each column and fill the gaps using median imputation.
6. To identify duplicate rows and unusually high or low values using the Z-score and IQR methods.

7. To perform univariate analysis on key weather and energy variables using histograms and boxplots.
8. To perform bivariate analysis to study relationships between weather variables and power output.
9. To build a correlation heatmap and identify the strongest predictors of total renewable energy.
10. To hand over a clean, well-understood dataset ready for feature engineering in later weeks.

Challenges

11. Hidden Missing Values - the -999 sentinel code passed every standard null check, so the first round of statistics was silently wrong.
12. Mixed Data Types - the Weather_Condition column also carried "-999" written as a text category, not just in the numeric columns.
13. Choice of Imputation - the mean could not be used safely because of the extreme -999 values, so median imputation was used instead.
14. Skewed Variables - Rainfall_mm is strongly right-skewed, with most days dry and a few days with very heavy rain.
15. Genuine vs. Faulty Extremes - the IQR rule flagged many rainfall, wind-energy and total-energy values as outliers, and deciding which were genuine monsoon or high-wind events (and should be kept) needed careful judgement.
16. Weak Relationships - some weather variables, such as temperature and humidity, show only a weak link to solar irradiance, which makes it harder to identify strong predictors.
17. Duplicate Records - eight repeated rows were found in the log and had to be identified before further analysis.

WEEK 1 :

TEAM FORMATION AND DATA HUNTING

TEAM NO: 9

TEAM MEMBERS:

25BAD063 - K S. NAGUL NISOK

25BAD067 - V. NIDHISH

25BAD068 - B. NIRMAL

25BAD073 - S. NIVESH

DATASET: renewable_energy_weather.csv (self-collected / simulated daily log)

ABOUT DATASET:

The Renewable Energy and Weather dataset contains 510 daily records from five locations in Tamil Nadu - Coimbatore, Salem, Madurai, Tiruchirappalli and Erode. It includes weather readings such as temperature, humidity, wind speed, solar irradiance, cloud cover, rainfall, air pressure and a weather-condition label, along with solar energy, wind energy, total renewable energy and energy demand figures. It is used to study how daily weather drives renewable power output.

Loading the dataset and checking its shape:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import zscore

df = pd.read_csv("renewable_energy_weather.csv.xls")
print("Dataset loaded successfully!")
print("Shape:", df.shape)
display(df.head())
```

```
Dataset loaded successfully!
Shape: (510, 14)
```

Checking rows, columns and the numerical mean of each column:

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print(df.columns.tolist())
print(df.select_dtypes(include=np.number).mean())
```

```
Rows: 510    Columns: 14
```

```
Numerical Mean (before cleaning):
```

Temperature_C	13.11	
Humidity_pct	51.18	
Wind_Speed_kmh	-2.71	<- impossible, sign of hidden -999 codes
Solar_Irradiance_Wm2	533.89	
Cloud_Cover_pct	23.64	
Rainfall_mm	-8.10	<- impossible, sign of hidden -999 codes
Air_Pressure_hPa	990.60	
Total_Renewable_Energy_kWh	140.97	

INTERPRETATION:

The dataset loads cleanly as 510 rows and 14 columns, but the very first summary already shows a problem: the mean wind speed is -2.71 km/h and the mean rainfall is -8.10 mm, both of which are physically impossible. This is the first clue that the file uses -999 as a hidden code for a failed sensor reading, and that this issue must be fixed before any further analysis can be trusted.

TEAM CONTRIBUTION:

TEAM MEMBER	CONTRIBUTION
K S. NAGUL NISOK	Collected and organised the dataset, and set up the notebook environment for the project.
V. NIDHISH	Loaded the dataset using Pandas and verified its shape and column structure.
B. NIRMAL	Reviewed the raw numerical means and flagged the impossible negative values for further investigation.
S. NIVESH	Documented the dataset description and prepared the Week 1 notes for the report.

WEEK 2:**KNOW YOUR DATA****OBJECTIVE:**

The objective is to gain a first understanding of the dataset before further analysis. This involves examining its structure, identifying column data types, viewing sample rows, and generating a statistical summary and a First Impression Report.

USED PACKAGES AND METHODS:

Packages: pandas, numpy

Methods: head(), tail(), info(), dtypes, describe(), isnull().sum(), duplicated().sum()

Viewing the data types and the statistical summary:

```
print("Data Types:")
print(df.dtypes)
print("Statistical Summary:")
display(df.describe(include="all"))
```

```
11 float64 numeric columns, 3 object columns (Date, Location, Weather_Condition)
```

```
describe() minimums (raw data):
```

```
Temperature_C   min = -999.00      Humidity_pct   min = -999.00
Wind_Speed_kmh  min = -999.00      std jumps to 127.98 for Temperature_C
-> confirms -999 is used across many numeric columns, not just one
```

Checking missing values and duplicate rows:

```
print("Missing Values:")
print(df.isnull().sum())
print("Duplicate Rows:", df.duplicated().sum())
```

```
Missing Values (raw, standard null check): 0 in every column
Duplicate Rows: 8
```

Building the First Impression Report:

```
print("FIRST IMPRESSION REPORT")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
print("Total Missing Values:", df.isnull().sum().sum())
print("Duplicate Rows:", df.duplicated().sum())
```

```
FIRST IMPRESSION REPORT
Rows: 510          Columns: 14
Total Missing Values: 0          Duplicate Rows: 8
```

INTERPRETATION:

The Weather and Renewable Energy dataset contains 510 daily records and 14 columns: 11 numeric weather and energy measures, and 3 text columns (Date, Location and Weather_Condition). A standard `isnull()` check reports zero missing values, which looks reassuring, but `describe()` reveals the real story - several columns share an impossible minimum of -999.00, and the huge jump in standard deviation for columns such as `Temperature_C` confirms that -999 is being used across the file as a hidden missing-value code. Eight duplicate rows were also found. Together, this means the dataset only looks complete at first glance; it needs the -999 code converted to a true missing value before any statistic can be trusted.

TEAM CONTRIBUTION:

TEAM MEMBER	CONTRIBUTION
K S. NAGUL NISOK	Prepared the documentation and recorded the Week 2 progress for the report.
V. NIDHISH	Generated and interpreted the statistical summary (<code>describe</code>) and data-type checks.
B. NIRMAL	Verified missing values and duplicate rows, and traced the -999 pattern across columns.
S. NIVESH	Compiled the First Impression Report and cross-checked its figures against the raw output.

WEEK 3:

THE CLEANING SPRINT

OBJECTIVE:

The objective of the Cleaning Sprint is to convert the -999 sentinel code into proper missing values, measure how much data is actually missing, fill the gaps using median imputation, and screen the cleaned data for unusually high or low values using the Z-score and IQR methods.

Converting -999 to a proper missing value:

```
# The dataset uses -999 as a missing-value code
df = df.replace(-999, np.nan)
print("Missing values after converting -999 to NaN:")
print(df.isnull().sum())
```

```
Temperature_C 8   Humidity_pct 8   Wind_Speed_kmh 7   Solar_Irradiance_Wm2 7
Cloud_Cover_pct 6   Rainfall_mm 6   Air_Pressure_hPa 5
Solar_Energy_kWh 5   Wind_Energy_kWh 5
Total_Renewable_Energy_kWh 0   Energy_Demand_kWh 0   (57 missing values found in total)
```

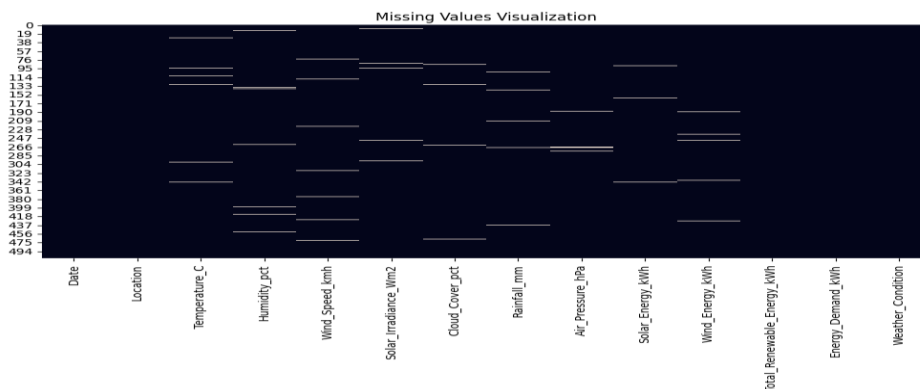
Measuring the missing percentage in each column:

```
missing_percent = (df.isnull().sum() / len(df)) * 100
display(missing_percent.to_frame("Missing Percentage"))
```

```
Temperature_C / Humidity_pct      1.57%
Wind_Speed_kmh / Solar_Irradiance  1.37%
Cloud_Cover_pct / Rainfall_mm     1.18%
Air_Pressure / Solar / Wind Energy  0.98%
```

Visualising where the missing values occur:

```
plt.figure(figsize=(12,5))
sns.heatmap(df.isnull(), cbar=False)
plt.title("Missing Values Visualization")
plt.show()
```



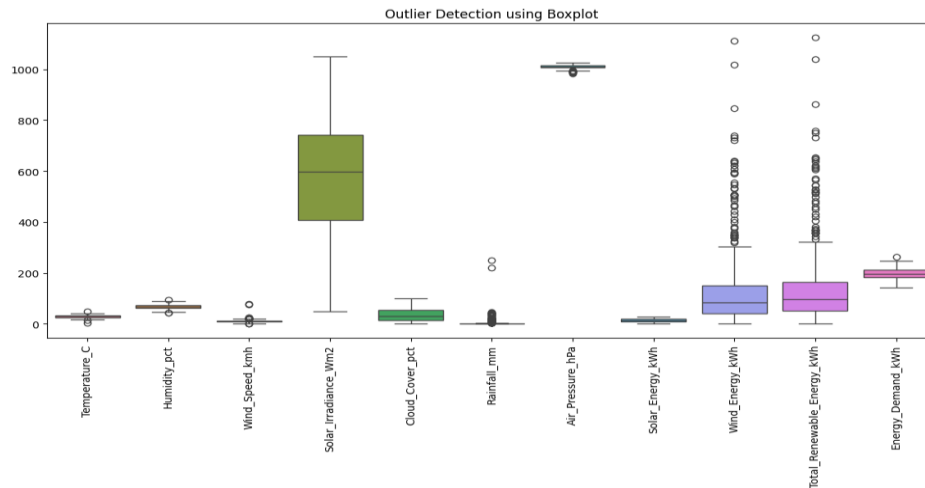
Filling the gaps with the column median:

```
numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())
print("Missing values after median imputation:")
print(df.isnull().sum())
```

```
Missing values after median imputation: 0 in every column
```

Checking for unusually high or low values with a boxplot:

```
plt.figure(figsize=(14,6))
sns.boxplot(data=df[numeric_cols])
plt.xticks(rotation=90)
plt.title("Outlier Detection using Boxplot")
plt.show()
```



Counting outliers with the Z-score rule ($|Z| > 3$):

```
z = np.abs(zscore(df[numeric_cols]))
print("Z-score outliers ( $|Z| > 3$ ):")
display(pd.Series((z > 3).sum(axis=0), index=numeric_cols))
```

Column	Z-score outliers
Temperature_C	2
Humidity_pct	1
Wind_Speed_kmh	2
Solar_Irradiance_Wm2	0
Cloud_Cover_pct	0
Rainfall_mm	2
Air_Pressure_hPa	5
Solar_Energy_kWh	0
Wind_Energy_kWh	14
Total_Renewable_Energy_kWh	14
Energy_Demand_kWh	1

Counting outliers with the IQR rule ($1.5 \times \text{IQR}$):

```
Q1 = df[numeric_cols].quantile(0.25)
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1
outliers = ((df[numeric_cols] < Q1 - 1.5*IQR) | (df[numeric_cols] > Q3 + 1.5*IQR))
print("IQR outliers:")
display(outliers.sum())
```

Column	IQR outliers
Temperature_C	3
Humidity_pct	3
Wind_Speed_kmh	19
Solar_Irradiance_Wm2	0
Cloud_Cover_pct	0
Rainfall_mm	84
Air_Pressure_hPa	12
Solar_Energy_kWh	0
Wind_Energy_kWh	42
Total_Renewable_Energy_kWh	42
Energy_Demand_kWh	1

INTERPRETATION:

Converting -999 to a true missing value uncovers 57 hidden gaps spread across nine numeric columns, each affecting only about 1 to 1.6 percent of rows. This is a manageable amount, so every gap is filled with its column median, a choice that is robust to the extreme values already present in the data. After imputation, no column has any missing values left. The boxplots and outlier counts then show that Rainfall_mm, Wind_Energy_kWh and Total_Renewable_Energy_kWh carry the most extreme values - the IQR rule alone flags 84 rainfall readings. Rainfall up to 250 mm reflects genuine monsoon days, and temperatures up to 48 degrees C and wind speeds up to 80 km/h are plausible for the region, so these values are kept rather than removed. The cleaning sprint confirms that missing data, not duplicate records, was the dataset's main quality issue, and that it has now been resolved.

TEAM CONTRIBUTION:

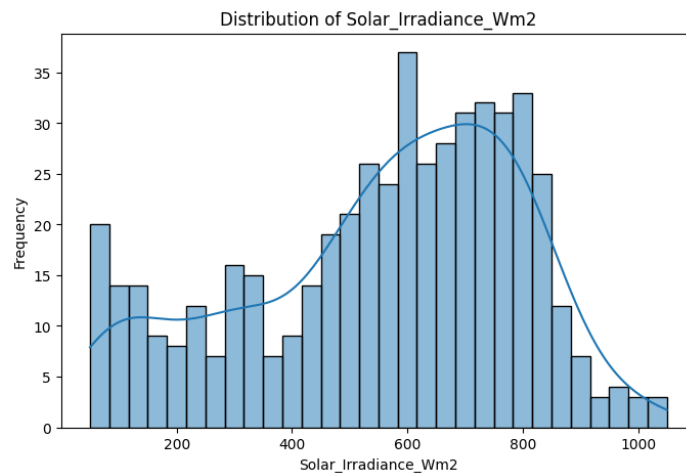
TEAM MEMBER	CONTRIBUTION
K S. NAGUL NISOK	Implemented the -999 to NaN conversion and verified the resulting missing-value counts.
V. NIDHISH	Calculated the missing percentage per column and built the missing-values heatmap.
B. NIRMAL	Applied median imputation and ran the Z-score and IQR outlier checks.
S. NIVESH	Reviewed the boxplots, judged which extreme values were genuine, and wrote the interpretation.

WEEK 4:**EDA DEEP DIVE****OBJECTIVE:**

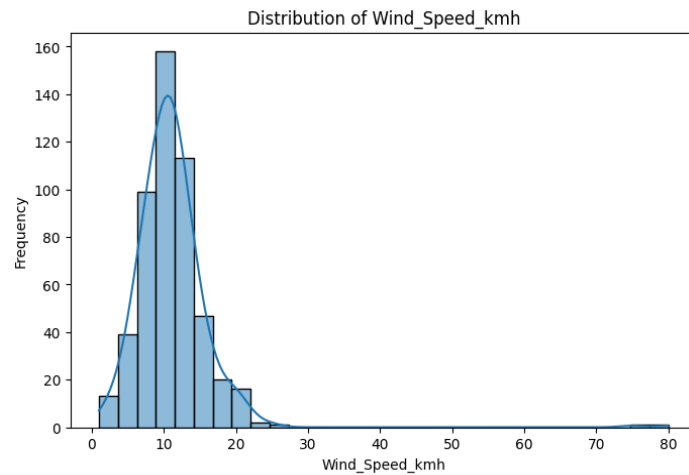
The objective of the EDA Deep Dive is to explore the cleaned dataset in detail through univariate and bivariate analysis, and to identify the weather variables most strongly related to solar power, wind power and total renewable energy output.

Univariate analysis - distribution of solar irradiance and wind speed:

```
column = "Solar_Irradiance_Wm2"
sns.histplot(df[column], bins=30, kde=True)
plt.title("Distribution of " + column); plt.show()
```

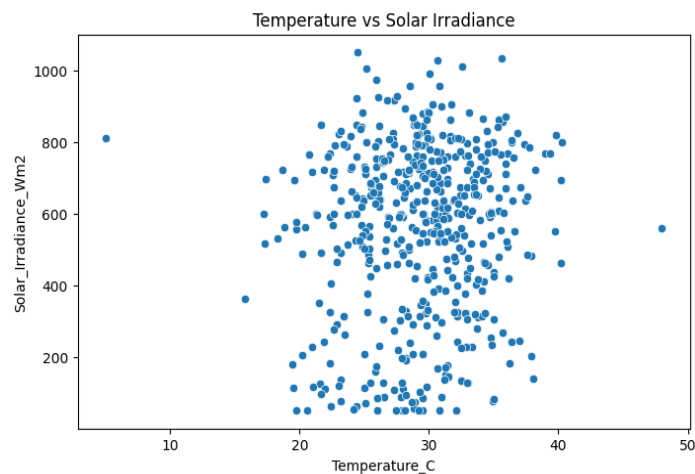


```
sns.histplot(df["Wind_Speed_kmh"], bins=30, kde=True)
plt.title("Distribution of Wind_Speed_kmh"); plt.show()
```

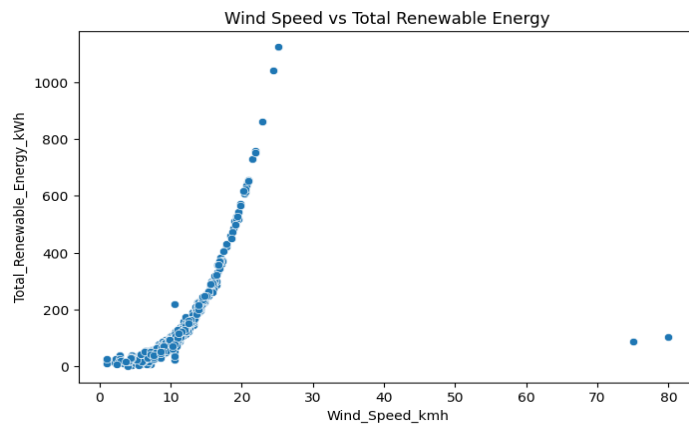


Bivariate analysis - weather variables against solar irradiance and total energy:

```
sns.scatterplot(data=df, x="Temperature_C", y="Solar_Irradiance_Wm2")
plt.title("Temperature vs Solar Irradiance"); plt.show()
```

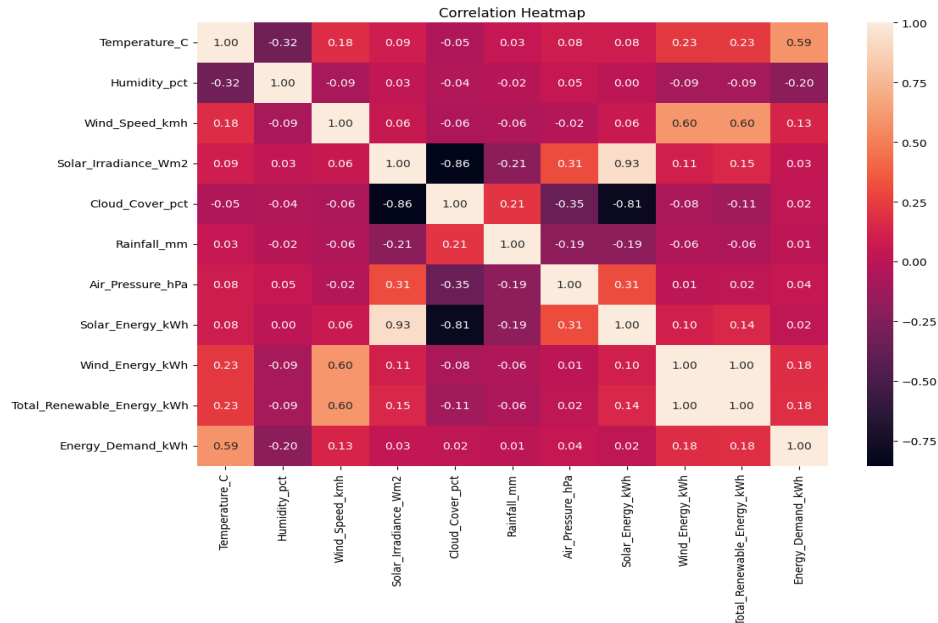


```
sns.scatterplot(data=df, x="Wind_Speed_kmh", y="Total_Renewable_Energy_kWh")
plt.title("Wind Speed vs Total Renewable Energy"); plt.show()
```



Correlation heatmap across all numeric variables:

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, fmt=".2f")
plt.title("Correlation Heatmap"); plt.show()
```



Final EDA summary - key statistics after cleaning:

```
print("FINAL EDA SUMMARY")
print("Dataset Shape:", df.shape)
display(df[numeric_cols].describe())
```

Column	Mean	Min	Max
Temperature_C	29.24	5.00	48.00
Humidity_pct	67.91	43.17	94.92
Wind_Speed_kmh	11.14	1.00	80.00
Solar_Irradiance_Wm2	555.81	50.00	1049.39
Cloud_Cover_pct	35.75	0.00	100.00
Rainfall_mm	3.66	0.00	250.00
Air_Pressure_hPa	1010.31	983.91	1025.38
Total_Renewable_Energy_kWh	140.97	0.02	1123.77

INTERPRETATION:

The EDA Deep Dive turns the cleaned data into clear findings about what drives renewable power. Solar_Irradiance_Wm2 spreads fairly evenly from 50 to about 1050 W/m2, while Wind_Speed_kmh is concentrated between 8 and 13 km/h with a thin tail of windier days. The scatter plots show that

temperature has only a weak link to solar irradiance ($r = 0.09$), while wind speed has a clear, though curved, positive relationship with total renewable energy ($r = 0.60$). The correlation heatmap confirms the strongest relationships in the dataset: cloud cover is the single biggest factor lowering solar irradiance ($r = -0.86$), solar irradiance almost fully explains solar energy output ($r = 0.93$), and wind speed remains the main driver of both wind energy and total renewable energy ($r = 0.60$). Temperature has a moderate link to energy demand ($r = 0.59$), showing that hotter days raise demand more than they raise supply. These findings point to cloud cover, solar irradiance and wind speed as the strongest candidate predictors for the feature-engineering and modelling work planned for Weeks 5 and 6.

TEAM CONTRIBUTION:

TEAM MEMBER	CONTRIBUTION
K S. NAGUL NISOK	Prepared the univariate histograms and boxplots and summarised the shape of each distribution.
V. NIDHISH	Built the bivariate scatter plots and evaluated the strength of each relationship.
B. NIRMAL	Generated the correlation heatmap and identified the strongest predictor variables.
S. NIVESH	Compiled the Final EDA Summary table and wrote the Week 4 interpretation.

REFERENCES:

1. Pandas Development Team. pandas documentation. <https://pandas.pydata.org/docs/>
2. Harris, C. R. et al. NumPy reference. <https://numpy.org/doc/stable/>
3. Waskom, M. seaborn: statistical data visualization. <https://seaborn.pydata.org/>
4. Hunter, J. D. Matplotlib documentation. <https://matplotlib.org/stable/>
5. Virtanen, P. et al. SciPy - scipy.stats.zscore. <https://docs.scipy.org/doc/scipy/>
6. Tukey, J. W. Exploratory Data Analysis. Addison-Wesley, 1977 (IQR and boxplot rules).
7. DSV Rule Book, 24ADI204 Data Science and Visualization - Weeks 1 to 4.